Cyprus University of Technology

# CSE 226
# Algorithms and Complexity

**Lecture 7**
**Dynamic Programming (Part 2)**
**Δυναμικός Προγραμματισμός (Μέρος 2)**

Herodotos Herodotou

*Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis*



## Recall: Dynamic Programming - Steps

1. Characterize the structure of an optimal solution
2. Recursively define the value of an optimal solution
3. Compute the value of an optimal solution in a bottom-up fashion
4. Construct an optimal solution from computed information

- Step 4 may be omitted, if only the optimal value is required (and not the actual solution)

## Recall: Dynamic Programming - DP requirements

Two requirements:

1. **Overlapping subproblems** (**Επικαλυπτόμενα υποπροβλήματα**)
   - A recursive algorithm for the problem would solve the same subproblems over and over
   - Dynamic-programming instead solves each subproblem once and then stores the solution in a table where it can be looked up when needed
2. **Optimal substructure** (**Βέλτιστη υποδομή**)
   - A problem exhibits optimal substructure if an optimal solution to the problem contains within it optimal solutions to subproblems
   - In dynamic programming, we build an optimal solution to the problem from optimal solutions to subproblems





## Example: Longest Common Subsequence (LCS): *Motivation*

- Strand of DNA: string over finite set {A,C,G,T}
  - Each element of set is a base: adenine, cytosine , guanine, or thymine
- Compare DNA similarities
  $S_1$ = ACCGGTCGAGTGCGCGGAAGCCGGCCGAA
  $S_2$ = GTCGTTCGGAATGCCGTTGCTCTGTAAA
- One measure of similarity (μέτρο ομοιότητας):
  - Find the longest string $S_3$ containing bases that also appear (not necessarily *consecutively* (*διαδοχικά*)) in $S_1$ and $S_2$
  - $S_3$ = GTCGTCGGAAGCCGGCCGAA

## Example: LCS
*Definitions*

- The sequence $Z = \langle z_1, z_2, \dots, z_k \rangle$ is a **subsequence** (**υπο-ακολουθία**) of $X = \langle x_1, x_2, \dots, x_m \rangle$ if $\exists \langle i_1, i_2, \dots, i_k \rangle$ (strictly increasing indices of $X$) such that $\forall j = 1,2, \dots, k, x_{i_j} = z_j$
  - Example: $Z = \langle B, C, D, B \rangle$ is a subsequence of $X = \langle A, B, C, B, D, A, B \rangle$ with index sequence $\langle 2,3,5,7 \rangle$
- $Z$ is **common subsequence** (**κοινή υποακολουθία**) of $X$ and $Y$ if $Z$ is α subsequence of both $X$ and $Y$
  - Example: $X = \langle A, B, C, B, D, A, B \rangle$ and $Y = \langle B, D, C, A, B, A \rangle$
    - common subsequence but not longest $\langle B, C, A \rangle$
    - common subsequence. Longest? $\langle B, C, B, A \rangle$
- **Longest Common Subsequence** (**Μέγιστη Κοινή Υποακολουθία**) **Problem**: Given 2 sequences $X$, $Y$, find maximum-length common subsequence $Z$.





## Example: LCS
*Definitions*

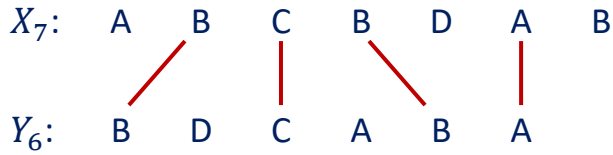
- Consider the following strings

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| $X_7$: | A | B | C | B | D | A | B |
| $Y_6$: | B | D | C | A | B | A | |

- Can you spot some common subsequences?
- Can you find a longest common subsequence (LCS)?
- What is the length of an LCS between x and y?
- Can you find one more LCS?

## Example: LCS
*Definitions*

- Here is an LCS




- The sequence <BCBA>
  - is a subsequence of both x and y
  - has maximal length (no longer common subsequence can be found)





## Example: LCS
*Brute Force Algorithm*

- Brute force algorithm:
  - Check every subsequence of $X_m = \langle x_1, x_2, \ldots, x_m \rangle$ to see if it is also a subsequence of $Y_n = \langle y_1, y_2, \ldots, y_n \rangle$
- Analysis of the brute-force algorithm
  - How much time does each checking take?
    - $O(n)$
  - How many subsequences of $X_m$ are there (to be checked)?
    - $2^m$
  - What is the worst-case running time?
    - $O(n\, 2^m)$

## Example: LCS
### *Step 1: Characterize an LCS*

**Theorem 15.1: Optimal LCS Substructure (Βέλτιστη Υποδομή LCS)**
Given sequences: $X_m = \langle x_1, x_2, \dots, x_m \rangle$ and $Y_n = \langle y_1, y_2, \dots, y_n \rangle$
For any LCS $Z_k = \langle z_1, z_2, \dots, z_k \rangle$ of $X_m$ and $Y_n$:
1. If $x_m = y_n$, then $z_k = x_m = y_n$ and $Z_{k-1}$ is an LCS of $X_{m-1}$ and $Y_{n-1}$
2. If $x_m \neq y_n$, then $z_k \neq x_m$ implies that $Z_k$ is an LCS of $X_{m-1}$ and $Y_n$
3. If $x_m \neq y_n$, then $z_k \neq y_n$ implies that $Z_k$ is an LCS of $X_m$ and $Y_{n-1}$

$X_5$: A B C B D

$Y_5$: B D C A B

$Z_3$: B C B





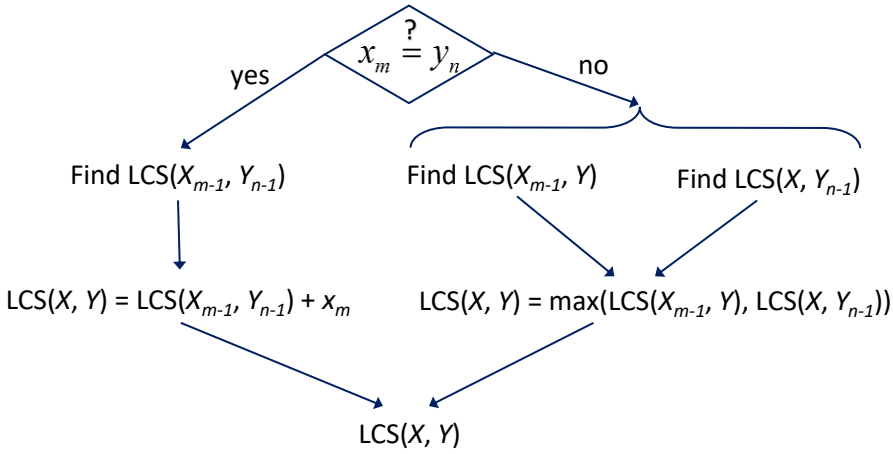
## Example: LCS
### *Step 1: Characterize an LCS*

**Theorem 15.1: Optimal LCS Substructure (Βέλτιστη Υποδομή LCS)**
Given sequences: $X_m = \langle x_1, x_2, \dots, x_m \rangle$ and $Y_n = \langle y_1, y_2, \dots, y_n \rangle$
For any LCS $Z_k = \langle z_1, z_2, \dots, z_k \rangle$ of $X_m$ and $Y_n$:
1. If $x_m = y_n$, then $z_k = x_m = y_n$ and $Z_{k-1}$ is an LCS of $X_{m-1}$ and $Y_{n-1}$
2. If $x_m \neq y_n$, then $z_k \neq x_m$ implies that $Z_k$ is an LCS of $X_{m-1}$ and $Y_n$
3. If $x_m \neq y_n$, then $z_k \neq y_n$ implies that $Z_k$ is an LCS of $X_m$ and $Y_{n-1}$

**Proof by contradiction** (Εις άτοπον απαγωγή)
1. a) Suppose $z_k \neq x_m$. Appending $x_m = y_n$ to $Z_k$ creates $Z_{k+1}$, which contradicts longest nature of $Z_k$
   b) Suppose common subsequence $W$ of $X_{m-1}$ and $Y_{n-1}$ has length $> k-1$. Appending $x_m$ to $W$ yields common subsequence $W'$ of $X_m$ and $Y_n$ of length $> k$ => contradiction
2. Common subsequence $W$ of $X_{m-1}$ and $Y_n$ of length $> k$ is also common subsequence of $X_m$ and $Y_n$, contradicting longest nature of $Z_k$
3. Similar to proof of (2)

## Example: LCS
### *Step 2: A Recursive Solution*

- Implications of **Theorem 15.1**:



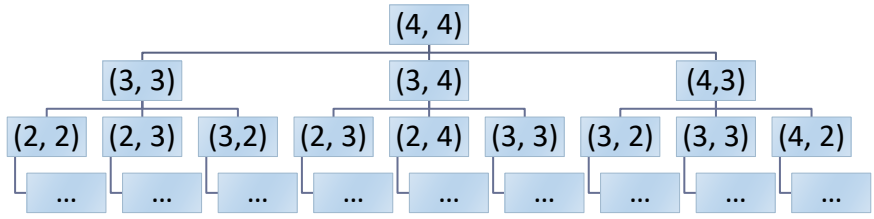





## Example: LCS
### *Step 2: A Recursive Solution*

- **Overlapping subproblems** (**Επικαλυπτόμενα υποπροβλήματα**):

$$LCS(X_{m-1}, Y_{n-1}) \left.\begin{array}{l} \subseteq LCS(X_{m-1}, Y) \\ \subseteq LCS(X, Y_{n-1}) \end{array}\right\} \subseteq LCS(X, Y)$$



## Example: LCS

*Step 2: A Recursive Solution*

- **Overlapping subproblems** (**Επικαλυπτόμενα υποπροβλήματα**):

$$LCS(X_{m-1}, Y_{n-1}) \left.\begin{array}{l} \subseteq LCS(X_{m-1}, Y) \\ \subseteq LCS(X, Y_{n-1}) \end{array}\right\} \subseteq LCS(X, Y)$$

- Θ(*mn*) distinct subproblems
- Let $c[i,j]$ denote the length of LCS of $X_i$ and $Y_j$. Recurrence for *length* of **optimal solution** (**βέλτιστη λύση**):

$$c[i,j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0 \\ c[i-1,j-1]+1 & \text{if } i,j > 0 \text{ and } x_i = y_j \\ \max(c[i,j-1], c[i-1,j]) & \text{if } i,j > 0 \text{ and } x_i \neq y_j \end{cases}$$

- Conditions of problem can exclude some subproblems!





## Example: LCS

*Step 3: Compute Length of an LCS*

```
LCS-LENGTH(X, Y)
 1  m = X.length
 2  n = Y.length
 3  let b[1..m, 1..n] and c[0..m, 0..n] be new tables
 4  for i = 1 to m
 5      c[i, 0] = 0
 6  for j = 0 to n
 7      c[0, j] = 0
 8  for i = 1 to m
 9      for j = 1 to n
10          if x_i == y_j
11              c[i, j] = c[i − 1, j − 1] + 1
12              b[i, j] = "↖"
13          elseif c[i − 1, j] ≥ c[i, j − 1]
14              c[i, j] = c[i − 1, j]
15              b[i, j] = "↑"
16          else c[i, j] = c[i, j − 1]
17              b[i, j] = "←"
18  return c and b
```

| i | | j | 0 | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|---|---|---|
| | | $y_j$ | | B | D | C | A | B | A |
| 0 | $x_i$ | | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | A | | 0 | ↑ 0 | ↑ 0 | ↑ 0 | ↖ 1 | ← 1 | ↖ 1 |
| 2 | B | | 0 | ↖ 1 | ← 1 | ← 1 | ↑ 1 | ↖ 2 | ← 2 |
| 3 | C | | 0 | ↑ 1 | ↑ 1 | ↖ 2 | ← 2 | ↑ 2 | ↑ 2 |
| 4 | B | | 0 | ↖ 1 | ↑ 1 | ↑ 2 | ↑ 2 | ↖ 3 | ← 3 |
| 5 | D | | 0 | ↑ 1 | ↖ 2 | ↑ 2 | ↑ 2 | ↑ 3 | ↑ 3 |
| 6 | A | | 0 | ↑ 1 | ↑ 2 | ↑ 2 | ↖ 3 | ↑ 3 | ↖ 4 |
| 7 | B | | 0 | ↖ 1 | ↑ 2 | ↑ 2 | ↑ 3 | ↖ 4 | ↑ 4 |

## Example: LCS
### *Step 4: Construct an LCS*

- Begin at $b[m, n]$ and trace through the table following the arrows

```
Print-LCS(b, X, i, j)
    if i == 0 or j == 0
        return
    if b[i, j] == "↖"
        Print-LCS(b, X, i − 1, j − 1)
        print X[i]
    else if b[i, j] == "↑"
        Print-LCS(b, X, i − 1, j)
    else
        Print-LCS(b, X, i, j − 1)
```

- What is the complexity of this method?





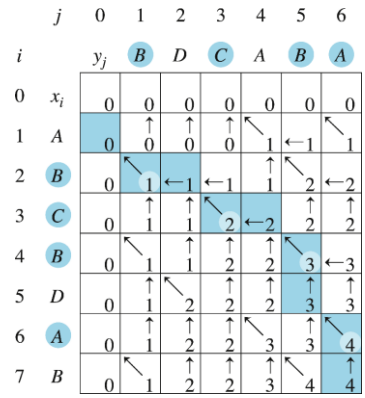
## Example: LCS
### *Improve the Code*

- **Eliminate $b$ table** (**Διαγραφή πίνακα $b$**)
  - $c[i, j]$ depends only on 3 other $c$ table entries:
    - $c[i-1, j-1]$, $c[i-1, j]$, $c[i, j-1]$
    - given value of $c[i, j]$, can pick the one in $O(1)$ time
    - reconstruct LCS in $O(m+n)$ time similar to PRINT-LCS
    - Same $\Theta(mn)$ space, but $\Theta(mn)$ was needed anyway. *Why?*
- **Asymptotic space reduction** (**Ασυμπτωτική μείωση χώρου**)
  - need only 2 rows of $c$ table at a time
    - row being computed
    - previous row
  - but does not preserve LCS reconstruction data

## Exercises

1. Show how to reconstruct an LCS from the completed $c$ table and the original sequences $X$ and $Y$ in $O(m+n)$ time, without using the $b$ table.

```
Print-LCS(c, X, Y, i, j)
    if i == 0 or j == 0
        return
    if X[i] == Y[j]
        Print-LCS(c, X, Y, i − 1, j − 1)
        print X[i]
    else if c[i − 1, j] ≥ c[i, j − 1]
        Print-LCS(c, X, Y, i − 1, j)
    else
        Print-LCS(c, X, Y, i, j − 1)
```








## DP guidelines: Finding Optimal Substructures

1. Show that a solution to the problem consists of making a choice, which results in one or more subproblems to be solved
2. Suppose you are given a choice leading to an optimal solution
3. Determine which subproblems follow and how to characterize the resulting space of subproblems
4. Show that the solution to the subproblems used within the optimal solution to the problem must themselves be optimal. Proof by contradiction is useful:
   - Suppose that each of the subproblem solutions is not optimal and then derive a contradiction by "cutting out" the nonoptimal subproblem solution and "pasting in" the optimal one

## DP guidelines: Constructing an optimal solution

- Dynamic programming uses optimal substructure **bottom up**:
  - *First* find optimal solutions to subproblems.
  - *Then* choose which to use in optimal solution to the problem
- Use an auxiliary table:
  - Store the choice of the subproblem in each step
  - Reconstruct the optimal solution from this table





## Counter-example: Unweighted Longest Simple Path

- The Unweighted Longest Simple Path (ULSP) problem:
  - Find a simple path (i.e., without cycles) from $u$ to $v$ consisting of the most edges
- Does it have the optimal substructure property?
  - Consider the ULSP, $u \overset{p}{\rightsquigarrow} v$ and let $w$ be an intermediate node, i.e., $u \overset{p_1}{\rightsquigarrow} w \overset{p_2}{\rightsquigarrow} v$
  - Are paths $p_1$ and $p_2$ necessarily optimal? That is, is $p_1$ the ULSP between $u$ and $w$, and is $p_2$ the ULSP between $w$ and $v$?
  - The answer is NO! Consider this example:
    - $q \to r \to t$ is the ULSP between $q$ and $t$
    - BUT, $q \to r$ is not the ULSP between $q$ and $r$
    - Similarly, $r \to t$ is not the ULSP between $r$ and $t$

